11 Query Builder and Repository Pattern

11.1 Introduction to Query Builder

When working with databases in Laravel, developers are provided with multiple approaches to interact with data. One of the most powerful and flexible tools Laravel offers is the **Query Builder**. Unlike raw SQL queries that are tightly coupled to the database syntax, the Query Builder provides a fluent and expressive interface for constructing and executing SQL statements in a programmatic way.

Laravel's Query Builder is built on top of PDO and supports all major database systems supported by Laravel, including MySQL, PostgreSQL, SQLite, and SQL Server. With its fluent interface, developers can chain methods together to construct complex queries that are still readable and maintainable. This is especially useful in scenarios where you want fine-grained control over SQL queries but still benefit from the safety and structure that Laravel provides.

One key advantage of using the Query Builder is its ability to prevent SQL injection attacks through automatic parameter binding. When you use methods like `where()` or `insert()`, Laravel ensures that all inputs are properly escaped and bound, reducing the security risks that often come with raw SQL queries.

Another benefit of the Query Builder is that it sits comfortably between using raw SQL and Laravel's Eloquent ORM. While Eloquent provides a full-featured, model-centric approach to working with data, it may sometimes be too heavy or abstract for simple queries or performance-critical code. Query Builder gives developers a leaner alternative that can be used for custom reporting, complex joins, or when avoiding the overhead of Eloquent models is preferred.

In this chapter, we will explore how to use Query Builder for common database operations such as selecting, inserting, updating, and deleting records. We will also demonstrate how Query Builder can be used effectively in combination with the Repository Pattern to create clean, reusable, and testable code that separates business logic from data access logic.